Backend pour le suivi des élèves - Step3

Prérequis

- Route
- Contrôleur
- ORM

Objectif

Nous allons maintenant créer nos 1ères routes et nos 1ers contrôleurs afin de mettre en place le CRUD des élèves.

Mise en place d'une route exemple

Pour commencer en douceur pour la mise en place des routes, je vous propose de créer une route qui aura seulement un but pédagogique et de test.

Ouvrez le fichier `start/routes.ts`.

> ⚠️ **ATTENTION**
> Ce fichier contient déjà des routes créées par le kit starter (`/api/v1/auth/signup` , `/api/v1/auth/login` , ...).
> **Ne les supprimez pas** : nous les utiliserons lorsque nous mettrons en place l'authentification.

A la fin du fichier `start/routes.ts`, nous ajoutons la route suivante :

```
// Route de test
router.get('test', async () => {
  return 'API is working!'
})
```

Pour accéder à cette route, nous devons faire un appel HTTP : `GET /test`

Nous allons tenter d'accéder à cette route depuis notre navigateur.

Pour ce faire nous devons exécuter notre serveur de développement grâce à la commande :

```
npm run dev
```

Puis, à l'aide de votre navigateur préféré, vous devez vous rendre à l'URL : `http://localhost:3333/test`

Le message : `API is working!` devrait s'afficher !

Maintenant que vous avez compris par la pratique ce qu'est une route, nous pouvons mettre en place le CRUD d'un élève.

Mise en place du contrôleur `Student`

> 📝 **À noter**
> Laissez tourner le serveur de développement (`npm run dev`) et ouvrez un 2ème terminal pour les commandes suivantes.

Création du contrôleur `Student` :

```
node ace make:controller student --api
```

Cette commande crée le fichier `app/controllers/students_controller.ts` .

> 📝 **À noter**
> AdonisJS met automatiquement le nom du contrôleur au pluriel : `student` devient `StudentsController` .

> 📝 **À noter**
> L'option `--api` crée un contrôleur ressource avec les 5 méthodes nécessaires à une API REST : `index` , `store` , `show` , `update` et `destroy` .
> Il existe aussi l'option `--resource` qui crée en plus les méthodes `create` et `edit` .
> Dans une application web classique, ces 2 méthodes ont pour but d'afficher les formulaires de création ou de modification d'un élève.
> Dans le cas d'une API REST, nous n'en avons pas besoin : c'est le front-end qui aura la responsabilité de gérer les formulaires.

Mise en place des routes

Avant d'écrire les routes, il faut comprendre comment AdonisJS 7 trouve nos contrôleurs.

Lorsque le serveur de développement tourne, AdonisJS génère automatiquement le fichier `.adonisjs/server/controllers.ts` .
Ce fichier recense tous les contrôleurs présents dans le dossier `app/controllers` :

```
export const controllers = {
  AccessTokens: () => import('#controllers/access_tokens_controller'),
  NewAccount: () => import('#controllers/new_account_controller'),
  Profile: () => import('#controllers/profile_controller'),
  Students: () => import('#controllers/students_controller'),
}
```

> 📝 **À noter**
> Ce fichier est mis à jour automatiquement à chaque ajout ou suppression d'un contrôleur. Il ne faut **jamais** le modifier à la main.
> Si `controllers.Students` est souligné en rouge dans votre éditeur, vérifiez que le serveur de développement tourne bien.

Ainsi, dans `start/routes.ts`, nous pouvons utiliser `controllers.Students` sans importer nous-mêmes le contrôleur.

Le début du fichier contient déjà l'import nécessaire :

```
import { controllers } from '#generated/controllers'
```

A la fin du fichier `start/routes.ts`, nous ajoutons les routes des élèves :

```
// Routes pour le CRUD /students
router.resource('students', controllers.Students).apiOnly()

/* est équivalent à :
router.group(() => {
  router.get('', [controllers.Students, 'index'])
  router.get(':id', [controllers.Students, 'show'])
  router.post('', [controllers.Students, 'store'])
  router.put(':id', [controllers.Students, 'update'])
  router.patch(':id', [controllers.Students, 'update'])
  router.delete(':id', [controllers.Students, 'destroy'])
}).prefix('students')
*/
```

On peut voir l'ensemble des routes avec la commande :

```
node ace list:routes
```

Retourner tous les élèves

Dans le fichier `app/controllers/students_controller.ts`, nous complétons la méthode `index` :

```
import Student from '#models/student'
import type { HttpContext } from '@adonisjs/core/http'

export default class StudentsController {
  /**
   * Afficher la liste des élèves
   */
```

```
  async index({}: HttpContext) {
    return Student.all()
  }

  // ...
}
```

Vous pouvez voir le JSON des élèves en visitant l'URL `http://localhost:3333/students`

> 💡 **Question**
> Que fait `Student.all()` ?

`Student` correspond à notre modèle.

`Student.all()` retourne un tableau d'élèves contenant tous les élèves présents dans la table `students`.

C'est l'ORM Lucid qui va se connecter à la db et faire la requête : `SELECT * FROM students`

Grâce à l'ORM Lucid, nous n'avons pas besoin d'écrire la requête SQL mais simplement `Student.all()`

> 💡 **Question**
> Pourquoi en faisant simplement `return Student.all()` cela retourne du JSON ?

En fait, Adonis va faire 2 choses automatiquement :

- pour le `return` Adonis va faire `ctx.response.send()`
- comme l'objet retourné par l'ORM Lucid (Model[]) est sérialisable, `ctx.response.send()` convertit automatiquement en JSON.

> 💡 **Problème**
> Résultat pas trié ?

Il y a un petit problème dans le json retourné. Ce serait plus agréable d'avoir les élèves ordonnés par le nom puis en cas d'homonyme par le prénom.

```
/**
 * Afficher la liste des élèves
 */
async index({}: HttpContext) {
  return Student.query().orderBy('name').orderBy('firstname')
}
```

Retourner un élève

```
/**
 * Afficher un élève
 */
async show({ params }: HttpContext) {
  return Student.findOrFail(params.id)
}
```

Vous pouvez voir le JSON de l'élève dont l'id est passé en paramètre en visitant l'URL `http://localhost:3333/students/3`

> 📝 **À noter**
> `params.id` va récupérer l'id présent dans la requête HTTP

> 💡 **Question**
> Que fait `Student.findOrFail(params.id)` ?

Cela va récupérer l'élève dont l'id=3 (dans notre cas).

Dit autrement, l'ORM Lucid va (en utilisant notre modèle `Student`) exécuter la requête `SELECT * FROM students WHERE id = 3`